

Part time Parliament 중간발표

이준서



목차

- Paxos 알고리즘 개요
- Part-time Parliament 소개
- Basic Paxos Protocol
- Complete Synod Protocol
- Multi Paxos (Multi-Decree Parliament)
 - Protocol
 - Further Developments
- Paxos의 한계
 - 추가과제

Paxos 알고리즘 개요



Paxos 알고리즘의 개발 배경

- Time, clocks, and the ordering of events in a distributed system[Lamport 1978]
 - 임의의 state machine 구현하는 첫 알고리즘
 - 모든 프로세스가 정상적으로 작동할 것을 가정
 - 단일 실패가 시스템 전체의 동기화에 영향을 미침
- Using time instead of timeout for fault-tolerant distributed system[Lamport 1984]
 - f 개의 임의의 실패에도 상태 기계의 명령은 고정된 시간 안에 실행됨
 - 하지만 f 개보다 더 많은 프로세스가 실패하면 다른 서버들은 inconsistent한 copies를 가짐
- Paxos Parliament's Protocol
 - 임의의 악의적인 실패를 허용하지 않고, bounded-time response도 보장하지 않음
 - 하지만 몇 번의 benign 실패라도 Consistency는 유지됨

Paxos 알고리즘의 목적

- 비동기 분산 시스템에서의 합의 문제 해결을 위함
 - **Consistency**
 - 모든 노드가 동일한 값을 합의하여 일관된 상태 유지
 - **Liveness**
 - 충분한 시간이 주어지면 모든 올바른 노드는 반드시 결정을 내림
 - **Partition Tolerance**
 - 네트워크 분할 발생해도 시스템이 계속 동작
 - **비동기 상황**
 - 메시지 지연
 - 독립적인 노드 실행
 - 시간 동기화 부재

Part-Time Parliament 소개



의회와 분산시스템 비유

➤ Part-time Parliament : fault-tolerant 분산 시스템

- Legislator : process
- leaving chamber : failure
- decree : 합의할 value
- passing decree : value is chosen
- ledgers : database

의회 프로토콜의 요구사항

- consistency of ledgers
 - 같은 번호의 decree가 다른 ledgers에서 모순된 내용이면 안됨
 - 공백인 것은 괜찮음
- progress condition
 - 의회에 majority 의원이 있고 긴 시간동안 들어오거나 나가지 않는다면, 그 자리에서 제안된 decree들은 통과된다.
 - 모든 통과된 법령은 의회에 있는 모든 의원의 ledger에 나타난다.

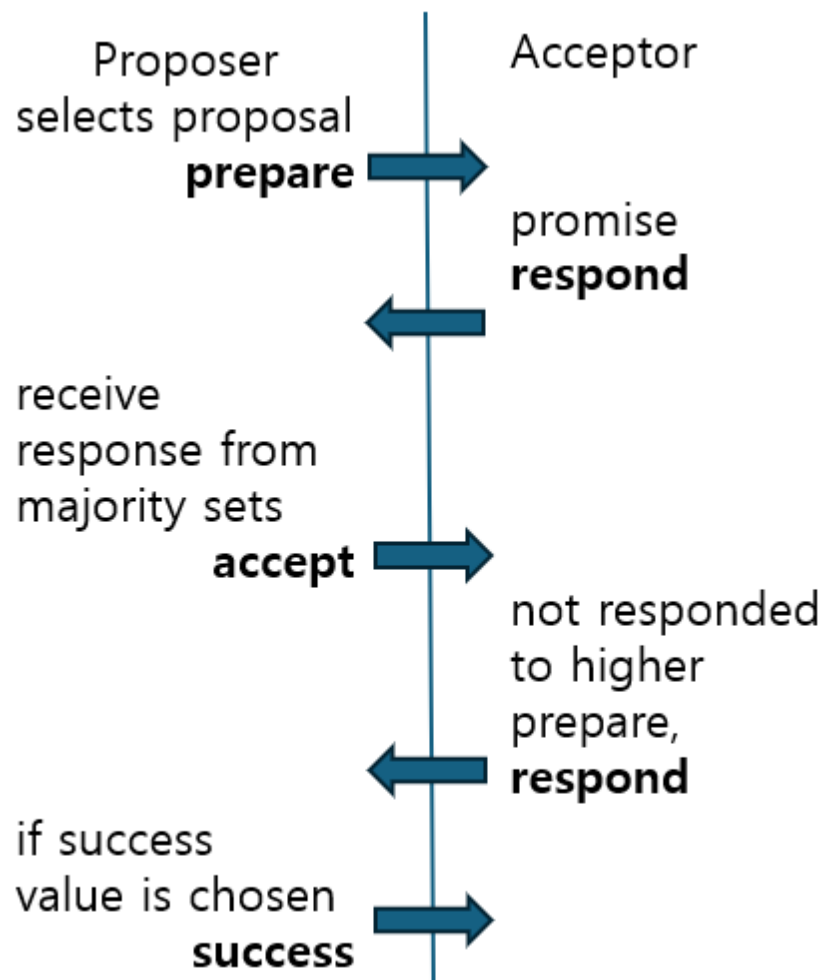
가상 의회 모델 구성요소

- **Proposer**
 - 새로운 법안(제안)을 제출
 - 의회에서 새로운 법안을 제출하는 의원으로 비유
- **Acceptor**
 - 제안된 법안을 수락하거나 거부
 - 의회에서 법안에 투표하는 의원들
- **Learner**
 - 최종적으로 합의된 값을 학습하고 기록
 - 의회에서 최종 결정을 기록하는 서기
 - 새롭게 learner을 두지 않고 Proposer와 Acceptor 모두가 Learner임

Basic Paxos



예비 protocol



consistency 보장하는 3가지 조건

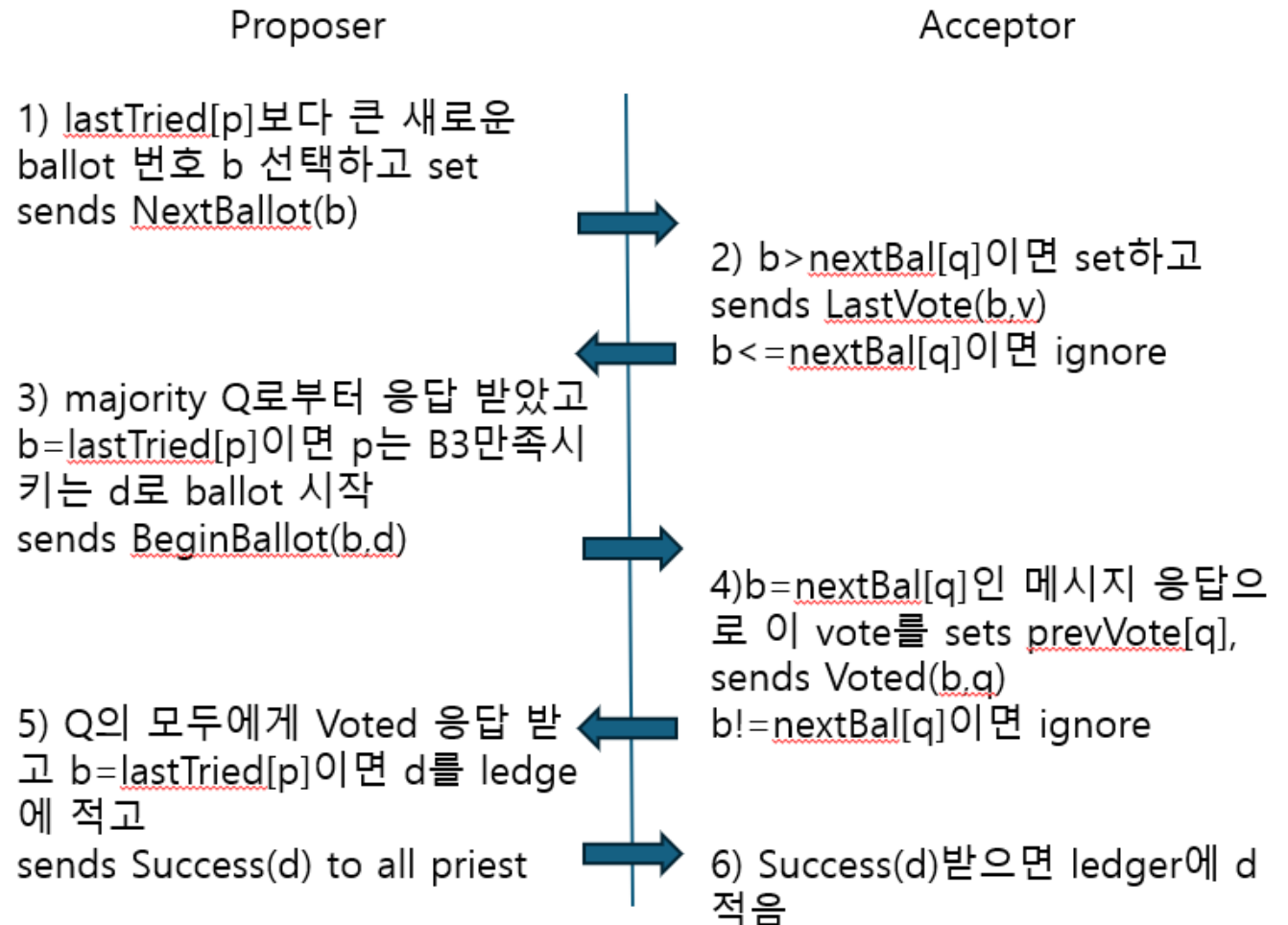
- B1(β): β 에 있는 ballot은 각각 고유 번호가 있다
- B2(β): β 의 두 quorums는 최소 한 명의 공통 구성원을 가진다
- B3(β): β 의 모든 ballot B에 대해 B의 quorum에 있는 구성원이 이전 ballot에서 투표했다면, B의 decree는 이전 ballots 중 가장 최근의 decree와 동일하다.

#	decree	quorum and voters				
2	α	A	B	Γ	Δ	
5	β	A	B	Γ		E
14	α		B		Δ	E
27	β	A		Γ	Δ	
29	β		B	Γ	Δ	

<Example for five ballots>

Basic Protocol

- **lastTried[p]**
 - p가 시작하려고 시도한 가장 최근의 ballot 번호
- **prevVote[p]**
 - p가 투표한 ballot 중 가장 높은 ballot 번호 가진 vote
- **nextBal[p]**
 - p가 보낸 LastVote 메시지 중 가장 큰 ballot 번호
- **LastVote(b,v)**
 - v는 MaxVote(b,q, β)
- **BeginBallot(b,d)**
 - d는 MaxVote(b,Q, β)의 decree

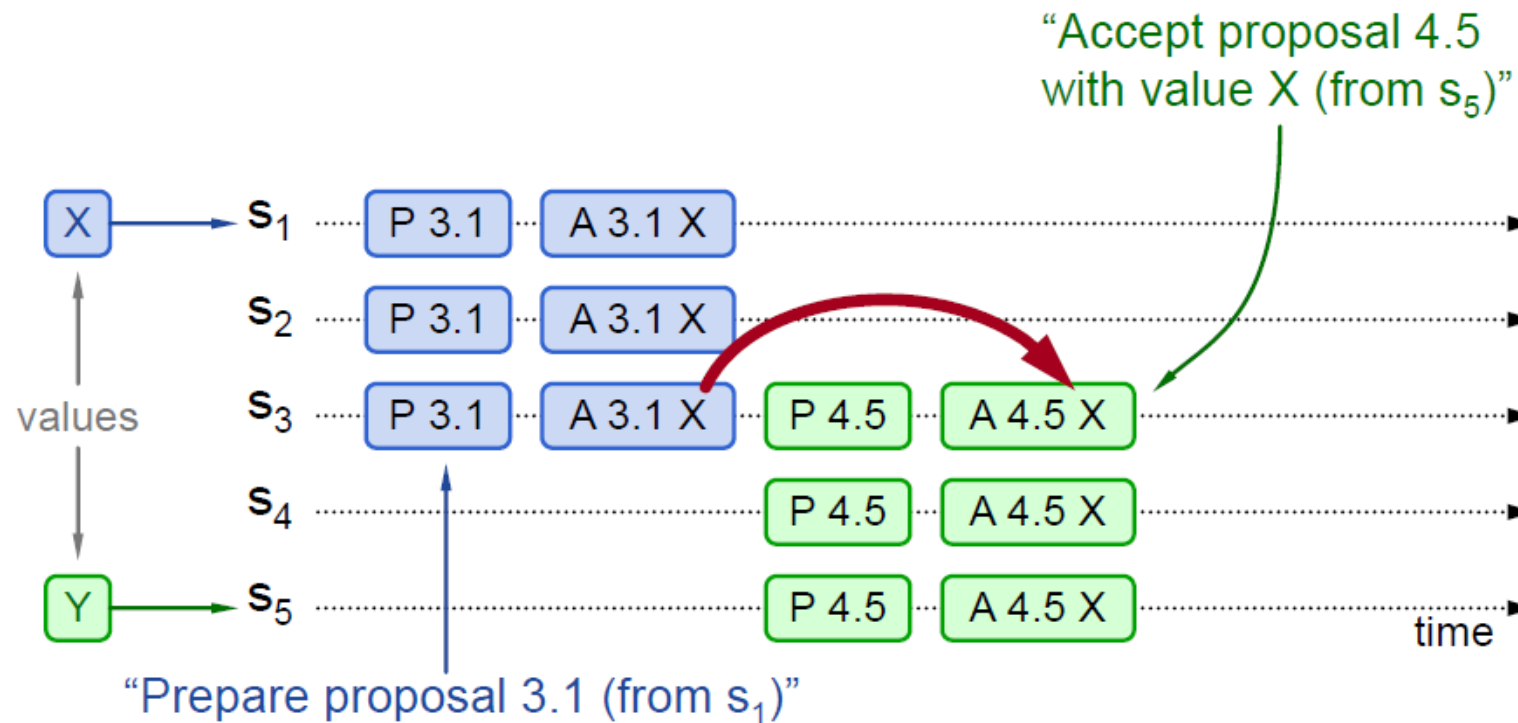


ballot이 경쟁적으로 대칭되는 상황

- 이전 decree가 이미 선택되었을 때
 - 프로토콜 대로 실행하고 B3에 의해서 decree 값 정해짐
- 이전 decree가 선택되지 않았지만 새로운 제안자가 봤을 때
 - 4번 step에서 ignore됨
- 이전 decree가 선택되지 않았고 새로운 제안자가 보지 못했을 때
 - 2번 step에서 ignore됨

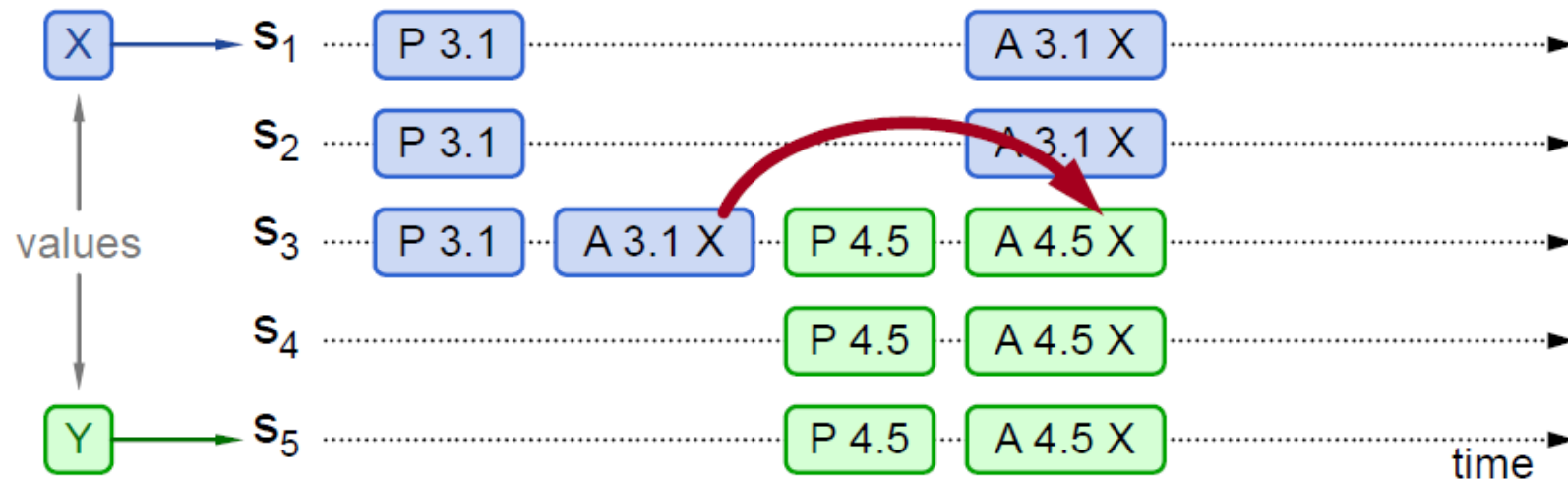
Previous value already chosen

- 새로운 제안자가 value값 찾고 사용할 것임



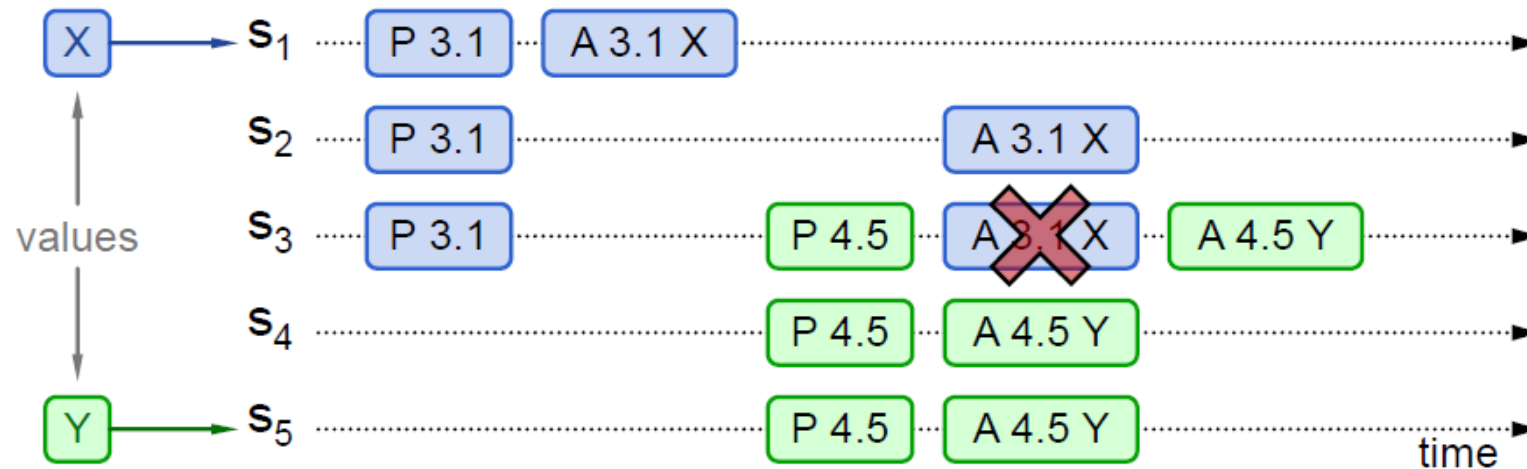
Previous value not chosen, but new proposer see it

- 새로운 제안자가 기존 value를 사용
- proposer 둘 다 successful



Previous value not chosen, new proposer doesn't see it

- 새로운 제안자가 own value를 선택
- 기존의 제안은 blocked 됨



basic paxos의 한계점

- B1~B3를 만족시키는 프로토콜이어서 consistency는 유지되지만 progress는 보장 불가
 - 2명의 제안자가 서로 ballot 번호가 증가하는 ballot을 주기적으로 시작하면 진행 불가능

Complete Synod Protocol



progress 방해

➤ progress condition

- 의회에 majority of legislators가 있고, 충분히 긴 시간동안 나가거나 들어오지 않는다면 의회에 있는 legislator가 제안한 decree는 통과할 것이고 통과한 decree는 의회에 있는 legislator의 ledger에 기록될 것이다.

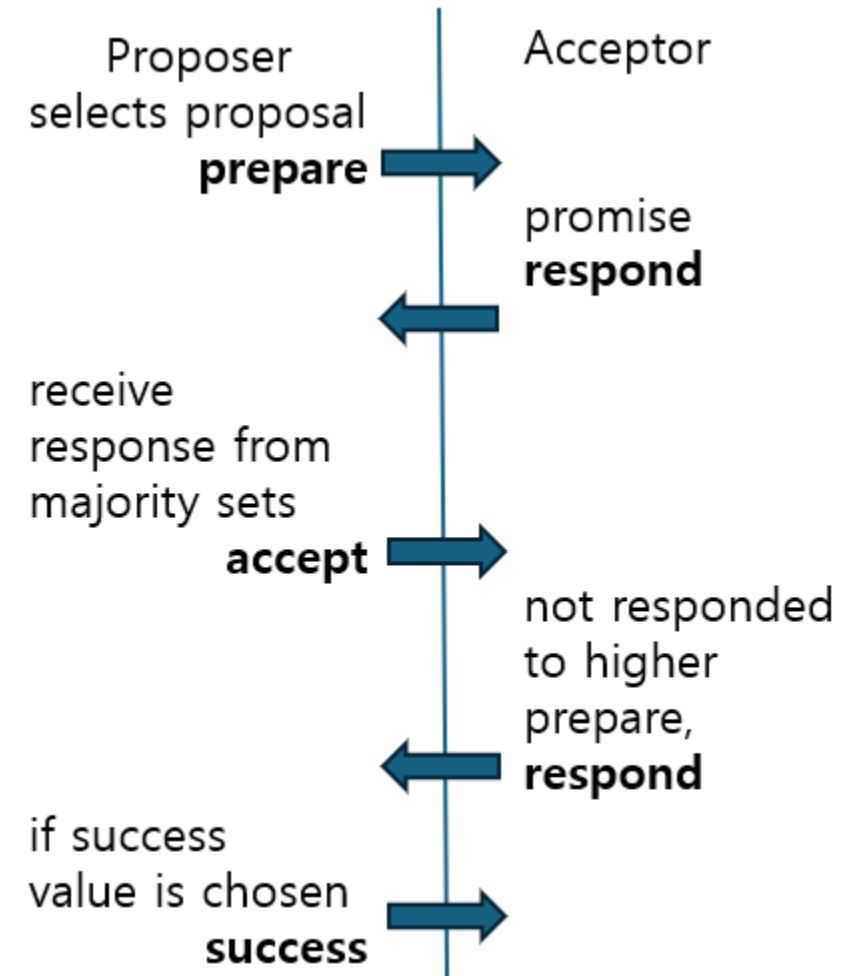
➤ 시작하지 않거나 너무 많은 ballot을 시작하는 것은 progress를 막음

➤ 하나의 succeed까지 시작하지만 너무 자주 시작하지는 않도록 함

- 자주 시작하면 이전 ballot들의 성공을 막음
- 메시지 전달, priest가 답할 때까지 걸리는 시간 알아야 함

프로토콜 진행에 걸리는 시간

- 메시지 전달 시간 4분
- legislator가 event처리하는데 걸리는 시간 7분
- step3 시작까지 22분
- step5 시작까지 22분
- 시간 내에 실행 안되는 경우
 - ballot시작하고 의회 나간 사람 있을 때
 - 더 큰 번호의 ballot이 다른 legislator에 의해 시작
- 후자의 경우 처리를 위해 acceptor가 respond할 때 더 큰 번호의 ballot 번호를 보냄



single leader로 progress 보장

- single legislator가 새로운 ballot 시작 요청 가정
 - step3나 step5를 이전 22분 안에 실행하지 않았을 때
 - 다른 proposer가 더 높은 번호의 ballot 시작했다는 것을 배웠을 때
- 의회의 문이 잠기고 majority의 legislator가 있으면 99분 안에 decree가 통과되고 ledger에 적힐 것
 - 다음 ballot 시작 22분
 - 다른 priest에 의해 더 높은 번호 ballot 시작했던 것을 learn 22분
 - step1~6까지 55분
- single proposer를 가정하면 progress 조건 보장

leader election protocol

- leader selection requirement
 - 의회 문 잠겼을 때, T분이 지나면 정확히 한 명의 구성원이 본인이 leader라고 생각할 것이다
- leader는 구성원 중 이름 순으로 정함
 - T-11분에 모든 legislator들에게 자신의 이름과 lastTried[p] 보냄
 - T분에 더 높은 이름의 legislator에게 메시지 받지 못했으면 leader
 - lastTried[p]도 같이 보내서 충분히 큰 ballot 번호로 시작 가능

basic protocol과의 차이점

- step 2~6을 최대한 빨리 실행
- ballot을 시작하는 leader 선출
- 적절한 시간에 leader는 ballot을 시작

Multi Paxos(Multi-Decree Parliament)



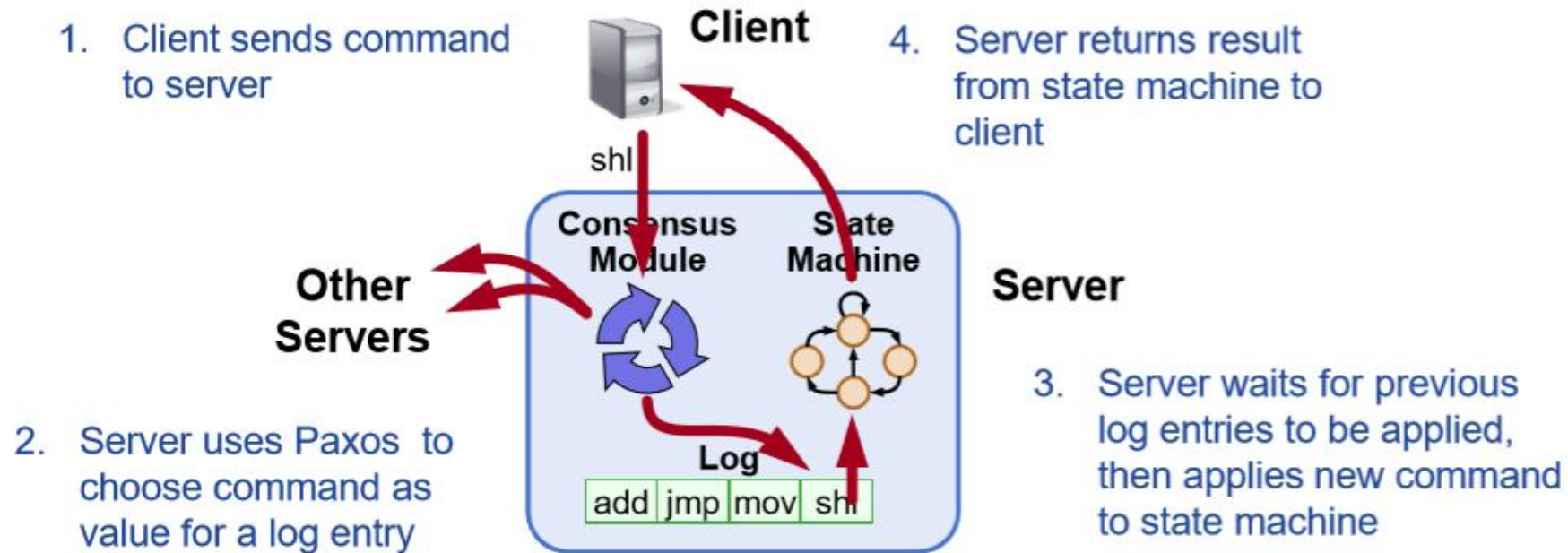
single paxos와의 차이점

- 단 하나의 decree만 선택하는 basic paxos와 다르게 여러 decree를 통과시켜야 함
 - 각 decree 번호마다 complete synod 프로토콜을 독립적인 instance로 취급

state machine 구현

➤ 분산 데이터베이스에 가상 의회 모델 적용

- legislator : server
- citizen : client
- 현재 law : state



Multi paxos 구현 issue

- Performance 최적화
- client command abort
- decree 순서

prepare 최적화

- step1,2를 모든 instance마다 진행하지 않고 한 번만 수행
 - 리더 선출마다 한 번씩만 수행
- NextBallot(b,n) 메시지로 chosen된 decree 파악 가능
 - n보다 같거나 작은 decree는 leader가 모두 알고 있음
 - 응답으로 acceptor가 chosen decree 보내줌
 - acceptor는 n보다 작은 decree 중 ledger에 없는 decree 보내달라고 요청

client command abort 문제

- decree pass하는 것은 client command commit하는 것
- leader 변경 또는 네트워크 장애 등의 이유로 client command abort될 수 있음
- prepare 단계에서 중단되면 다른 decree로 대신 수행 가능
- accept, commit 단계에서 중단되면 decree 번호는 할당되어 ledger에 빈자리 생김
 - 더 높은 번호의 decree들은 chosen되었지만 사용 불가능

client command abort 방법

- “olive-day”decree 사용
 - state machine에서 “no-op”과 같은 역할
- 빈 decree에 “olive-day”decree를 통과시켜서 더 높은 번호의 decree 사용 가능

decree ordering

- leader가 새로운 decree를 제안하기 전에 통과된 decree들을 학습함
 - prepare단계
- leader는 순서에 맞지 않게 decree를 제안하지 않음
- 프로토콜은 decree-ordering property를 만족함
 - 만약 decree A와 B가 “olive-day” decree가 아니고, B가 제안되기 전 A가 통과되었다면 A는 B보다 작은 decree 번호를 가진다.

Further Developments

- Picking a president
 - 이름 순서대로 leader를 뽑을 때 문제
- Long ledgers
 - decree를 바로 ledger에 적지 않고 모아서 적도록 함
 - 하나마다 commit하지 않고 batch 하고 commit
- Bureaucrats
 - split brain 상황에서 자원의 현재 소유자 선택
- Learning the law
 - fast read
- Dishonest legislators and honest mistakes
 - redundant decree로 self-stabilizing
- Choosing new legislators
 - 구성원 변경

Split brain 대처

- Split brain 정의
 - 분산 시스템에서 네트워크 분할로 인해 노드들이 서로 통신할 수 없게 되어 클러스터가 여러 개의 분리된 부분으로 나뉘지는 현상
- 공유 자원에 대한 소유자 변경 과정에서 문제 발생
- 제안한 시간과 임기를 정함
 - 원래 소유자의 임기 또는 decree의 제안 시간 중 늦은 시간을 기다리고 새로운 소유자 시작
 - 원래 소유자가 통신이 되지 않아도 안전성 보장
 - 소유권 포기 통신이 실패했을 때 safety를 보장

fast read

- slow read와 차이점
 - state machine의 “read” command를 통해서 이루어짐
 - fast read는 server 데이터베이스의 현재 버전을 읽어서 수행됨
- monotonicity condition
 - 만약 한 문의가 다음 문의보다 앞선다면, 두번째 문의는 첫번째보다 앞선 state of law를 볼 수 없다
- 이 조건으로 인해서 fast read가 slow read와 다른 값을 read할 수 없도록 보장함

구성원 변경

- 합의에 참여하는 서버들의 정보 변경
 - 현재의 과반수를 결정
- 구성 변경에 의해 발생하는 이슈
 - 구성 변경을 위해 decree 통과시키려고 할 때 구성의 과반수를 어떻게 결정할 것인지
 - circularity problem
- n 번째 decree를 통과시키려고 하면 $n-\alpha$ 번째의 구성 변경에 영향을 받음
 - $n-\alpha$ decree가 통과되기전까지 leader는 n 번째 decree를 시작하지 않음
 - 즉, 한번에 통과시킬 수 있는 decree 개수 α 개
 - $n-\alpha+1$ 부터 $n-1$ 까지 decree는 “olive-day” decree로 뛰어넘을 수도 있음
 - 바로 새로운 구성변경 적용 가능

Paxos 한계

복잡한 구현

- 이해와 구현이 어려움
- 여러 단계와 역할의 정확한 구현 어려움
- 실질적으로 사용하기 위해서는 여러 최적화가 필요함
 - 최적화가 추가적인 복잡성 초래
 - 최적화의 필요 부분만 언급하고 명쾌한 해결 방법은 제시되지 않음
- 리더 선출, self-stabilizing 에 대한 정확한 알고리즘이 제시되지 않았음

성능 문제

- 합의를 도출하기 위해 여러 라운드의 통신 및 네트워크 메시지가 필요
 - Proposer가 다수의 Acceptor와 통신
- 네트워크 트래픽을 유발
- 노드 수 증가함에 따라 메시지 전달과 합의 도출에 필요한 시간 늘어남

비동기 분산 시스템에서의 한계점

- 기본 paxos는 progress를 보장하고 있지 않기 때문에 메시지 전달 지연이나 노드의 실패가 구분될 수 없어서 합의가 무기한 지연될 수 있음
 - 무기한 지연되기 전에 다수로부터 합의를 받으면 되지만 못 받는 경우도 있음
 - complete synod protocol이나 multi-paxos에서는 timeout을 사용함

최종발표까지의 추가 과제

- Multi-paxos는 구현의 여지가 있어서 구현한 사례들
- FLP impossibility
- Raft